

Checkpoint 3 — Testing Report

TaskFlow due-date feature (task.py) | QA Testing Agent

RESULT: GATE GREEN — clear to raise the PR

21 passed 0 failed exit 0

1. What was tested

The Checkpoint 3 due-date feature: tasks may carry an optional due date; the API derives an **overdue** flag (a date-only value means end of that day); the list can be sorted by due date; and standard task behavior (create, read, update, delete, reset) still holds. Acceptance criteria were reverse-derived from the module's public interface, as no numbered paper spec was handed off with the code.

2. Gate conditions — all 5 met

#	Condition	Status
1	At least one test per acceptance criterion (20 criteria)	PASS
2	Edge-case test for a task with no due date	PASS
3	Edge-case test for a task due today	PASS
4	All tests executed against the implementation code	PASS
5	PR description drafted (changes / testing / limitations)	PASS

3. Acceptance criteria — traceability & results

ID	Acceptance criterion	Test	Result
AC1	create_task sets id, title, completed=False, dueDate (None if omitted)	test_create_task_defaults	PASS
AC2	IDs are unique, increment, and not reused after delete	test_ids_are_unique_and_not_reused_after_delete	PASS
AC3	get_task returns the task by id; None for unknown id	test_get_task_returns_match_or_none	PASS
AC4	get_tasks returns all stored tasks	test_get_tasks_returns_all	PASS
AC5	update_task sets completed; None for unknown id	test_update_task_sets_completed_and_handles_unknown	PASS
AC6	delete_task removes and returns True; False for unknown id	test_delete_task_removes_and_handles_unknown	PASS
AC7	reset clears the store and the id counter	test_reset_clears_store_and_id_counter	PASS
AC8	parse_due_date(None) returns None	test_parse_due_date_none_returns_none	PASS
AC9	parse_due_date(date-only) = end of day, UTC ISO	test_parse_due_date_date_only_is_end_of_day_utc	PASS
AC10	parse_due_date(datetime with 'Z') = canonical UTC ISO	test_parse_due_date_zulu_datetime_is_canonical_utc	PASS
AC11	parse_due_date(non-string) raises ValueError	test_parse_due_date_non_string_raises	PASS
AC12	parse_due_date(invalid string) raises ValueError	test_parse_due_date_invalid_string_raises	PASS
AC13	task_view attaches an 'overdue' flag	test_task_view_attaches_overdue_flag	PASS
AC14	overdue = True when due is strictly past and not completed	test_overdue_true_when_past_and_not_completed	PASS
AC15	overdue = False when the task is completed	test_overdue_false_when_completed	PASS
AC16	overdue = False when there is no due date	test_overdue_false_when_no_due_date	PASS
AC17	overdue = False when due date is in the future	test_overdue_false_when_future	PASS

ID	Acceptance criterion	Test	Result
AC18	task_list_view default order = insertion order	test_list_view_default_is_insertion_order	PASS
AC19	task_list_view(sort='dueDate'): asc, undated last, ties by id	test_list_view_sorted_by_due_date	PASS
AC20	task_view / task_list_view never mutate or reorder the store	test_views_do_not_mutate_store	PASS

4. Required edge cases — results

ID	Edge case	Expected behavior	Test	Result
EC1	Task with NO due date (omitted / None)	dueDate stored as None; task not overdue	test_overdue_false_when_no_due_date	PASS
EC2	Task due EXACTLY today (date.today())	End-of-day semantics -> not overdue during today	test_task_due_exactly_today_is_not_overdue	PASS

Note: EC1 also satisfies AC16, so the 20 criteria and 2 edge cases are covered by 21 distinct test functions.

5. Execution result

Run in the project with task.py present: `pytest test_task_checkpoint3.py`

Outcome: **21 passed, 0 failed, 21 total (exit code 0)**. In this offline sandbox the suite was executed via a pytest-compatibility shim because pytest could not be installed; the test bodies and implementation are the real ones. Run under pytest in CI for the official verdict.

6. Known limitations & findings (non-blocking)

1. **Spec confirmation needed** — criteria derived from code, not a paper spec. Confirm they match intent, especially "due today = not overdue." (Highest priority for the reviewer.)
2. **Timezone sensitivity** — date-only / naive datetimes normalize in the server's local timezone. Tests are deterministic, but behavior is environment-dependent; add a contract note.
3. **get_tasks() returns by reference** — callers can mutate the store directly. Out of scope for this checkpoint.
4. **create_task trusts pre-normalized input** — it does not call `parse_due_date` itself. Recommend an API-level integration test for the route boundary.

7. PR decision

The gate blocks a PR only if a test fails. All 21 tests pass, and every item above is non-blocking by definition. **Verdict: raise the PR** — asking the reviewer to confirm the "due today" semantics before merge, the one finding that would flip the decision if intended behavior differs.